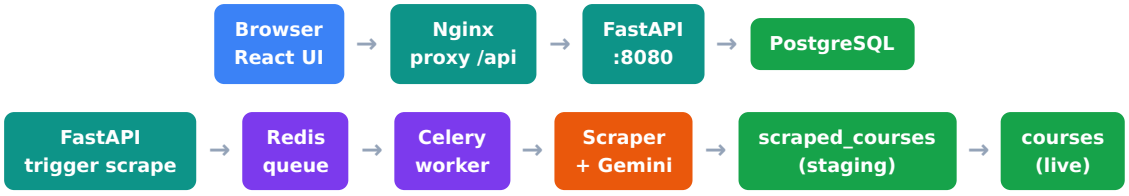
University Portal — System Architecture

Course / Fee / Intake & Requirement Management System | pnpm monorepo | FastAPI + React + Celery + PostgreSQL

A centralised admin portal for universities to manage course data — courses, fees, intakes, scholarships and admission requirements — with AI-powered web scraping, bulk Excel upload/download, and change detection. The system is a **pnpm monorepo**: a React/Vite frontend talks to a Python FastAPI backend (port 8080). Heavy scraping work runs asynchronously in Celery workers (Redis broker), with Gemini AI as a fallback extractor. All data is persisted in PostgreSQL.

End-to-end request & data flow



Layer 1 — Frontend

Browser / Admin Portal `artifacts/university-portal/`

Framework	React 18 + Vite, TypeScript 5.9
UI	shadcn/ui (Radix primitives) + Tailwind CSS
Data fetching	TanStack React Query — server-state caching, mutations, auto-refetch
Routing	Wouter — lightweight client-side SPA routing
Auth	HTTPOnly JWT cookie `session` (7-day). `AuthGuard` wraps all protected routes; `src/context/auth.tsx` exposes `useAuth()` + `AuthProvider`
Pages	Dashboard · Universities · Courses · Scraper Jobs · Import History · Scraped Changes
API comms	Relative URLs through the shared Nginx proxy; Orval-generated React Query hooks from the OpenAPI spec

How it works: On load, the app calls `GET /api/auth/me`. No valid session → redirect to `/login`. Each page uses React Query hooks to fetch from the API; mutations (create/edit/approve) invalidate the relevant query keys so the UI refreshes automatically.

Layer 2 — API Server (FastAPI)

FastAPI :8080 (Uvicorn)

- Python 3.12, ASGI via Uvicorn
- Dev: port 8080 (`--reload`)
- Prod: port 8000 behind Nginx
- Nginx: `/api → 127.0.0.1:8000`
- Entry: `backend-py/app/main.py`
- DB: asyncpg + SQLAlchemy (async)

Auth (JWT Cookie)

- `POST /api/auth/login` — sets cookie
- `POST /api/auth/logout` — clears cookie
- `GET /api/auth/me` — session check
- Cookie `session` (HTTPOnly, 7-day)
- Default `admin@university-portal.local`
- Override: `ADMIN_EMAIL` / `ADMIN_PASSWORD`

API Routers `backend-py/app/routers/`

courses.py	Course CRUD + bulk CSV export (with asyncpg NUMERIC→float decimal fix)
universities.py	University CRUD + associated course listing
scraper_jobs.py	Trigger scrape, poll job status, approve/reject scraped changes
bulk_import.py	Excel (.xlsx) upload → validation → `import_jobs` table
dashboard.py	Stats overview, courses by degree level, upcoming intakes, recent changes
change_history.py	Field-level diff log (before/after values) for the audit trail
auth.py	login / logout / me endpoints

Layer 3 — Background Workers

Celery Worker `app/tasks/`

- 4 concurrent workers, queue `-Q scrape`
- Broker + backend: Redis :6379
- Tasks: `run_scrape()`, `run_repair()`, `bulk_approve()`
- Start: `celery -A app.tasks.celery_app worker --concurrency=4 --beat`

Redis :6379 + Celery Beat

- Redis: in-memory broker only, no persistence (`--save ''`)
- Bind `127.0.0.1:6379`
- Beat: `nightly_sweep` @ 02:00 UTC
- → baseline snapshot → regression diff → drift alert

How background scraping works: The API enqueues a scrape job onto Redis. A Celery worker claims it, runs the full discovery + extraction pipeline, and writes results into the `scraped_courses` staging table. The nightly Beat task snapshots every university, diffs against the previous night, and fires a drift alert if unexpected field-value changes appear.

Layer 4 — Scraper Engine

Orchestrator — 7-tier discovery `orchestrator.py`

Tier 1	BFS crawl — breadth-first link crawl from the base URL
Tier 2	Sitemap — parse `sitemap.xml` / `sitemap_index.xml`
Tier 3	Alt-path probes — `/courses`, `/programs`, `/study`, `/international` …
Tier 4	Subdomain fallback — `handbook.<domain>`, `study.<domain>` (YAML-configured)
Tier 5	Playwright browser — headless Chrome for JS-heavy sites (RMIT, Deakin, UNSW)
Tier 6	Wayback Machine — archive.org fallback for blocked/broken sites
Tier 7	Discovery-failure alert — fires if <3 course links found after all tiers

Per-university YAML config is loaded at run start into a contextvar and deep-merged with global defaults.

Per-course Pipeline

- `stage_course.py` — write to `scraped_courses`
- Auto-publish gate — **85% completeness floor**
- 13 review fields: name, degree_level, category, mode, location, duration, intakes, fee, description, academic level/score, english_test, other_req
- `approve_course.py` — promote to `courses`
- `bulk_approve.py` — batch promotion
- Repair scrape — back-fill blank fields only (never overwrites)

Gemini AI — `gemini-2.5-flash-lite`

- AI fallback for hard-to-extract fields
- **Skip gate:** skip if ≥90% fields filled at ≥0.70 confidence (saves 30–50%)
- **Circuit breaker:** trips on 5 quota errors / 60s, stays open 5 min
- **Cost ceiling:** per-job USD budget cap
- **Call log:** `gemini_call_log` — tokens, cost, duration, success

Layer 5 — Field Extractors `app/services/scraper/extractors/`

Extractor	What it does
`course_name.py`	DOM title extraction, strips title suffixes / institution tails (driven by YAML `strip_title_suffixes`)
`duration.py`	DOM structural pass + sentence tournament; YAML `reject_sentence_patterns` filters bad text like "up to N years"
`fee.py`	International tuition extraction, PDF fee-table matching, CRICOS-matched rows, per-course PDF parsing
`english_test.py`	IELTS / PTE / TOEFL / CAE / DET band scores (overall + sub-bands); per-host PTE blocklist suppresses false positives
`location.py`	Campus city extraction, country-suffix append, online/virtual strip, slash-separated city normalisation; YAML `strip_patterns`
`intake.py`	Intake-month extraction; session→month map (Autumn→Mar, Spring→Jul, Summer→Nov for AU unis)
`cricos_code.py`	CRICOS code regex, PDF pipeline integration, AU universities only; provenance into `scraped_field_evidence`
`degree_level.py`	Degree-level + category classification
`description.py / eligibility.py`	Course description & eligibility extraction
`gemini_primary.py / ai_fallback.py`	AI-based extraction when rule-based extractors miss

Layer 6 — Per-University YAML Config System

scraper_config/ `backend-py/scraper_config/`

defaults.yaml	Conservative global defaults (changing needs a full regression sweep + human approval)
unis/<slug>.yaml	Per-university overrides — 30+ stubs (uwa, acap, bond, csu, monash …)
config/schema.py	Pydantic `UniConfig` split into `discovery` + `extraction` sections
config/loader.py	Deep-merge chain: defaults → DB `scrape_config` → per-uni YAML
config/context.py	`ContextVar[UniConfig]` scoped to each scrape job

Architecture rule: ALL per-university quirks live in `scraper_config/unis/<slug>.yaml` — never in global extractor code. Key fields: `strip_title_suffixes`, `reject_sentence_patterns`, `strip_patterns`, `domestic_only`, `fallback_subdomains`, `always_sitemap_supplement`. This means onboarding a new university needs zero global code changes.

Layer 7 — Database (PostgreSQL)

Core data tables

universities	name, website, country, scrape_config (JSONB)
courses	live courses: name, degree_level, fee, duration, location, mode
intakes	intake months per course
fees	fee breakdown per course
english_requirements	IELTS/PTE/TOEFL/CAE/DET bands
academic_requirements	GPA, score, level
scholarships	scholarship info per course

Scraping & ops tables

scraped_courses	staging: pending→review→approved→promoted
scraped_field_evidence	per-field provenance: method, source, confidence
scrape_runtime_jobs	job stats: found, staged, cost, elapsed
scrape_run_alerts	rule-based alerts per run
discovery_failure_alerts	Tier-7 fired alerts
gemini_call_log	every AI call: tokens, cost, success
import_jobs	Excel import history
change_history	field-level before/after audit log

SQL cost / coverage views

- `v_gemini_cost_by_university` — per-uni Gemini spend
- `v_gemini_cost_by_call_type` — spend by call type
- `v_gemini_top_spenders_30d` — top spenders last 30 days
- `v_gemini_skip_efficiency` — how often the skip gate fires
- `v_cricos_coverage_au` — CRICOS match rate per AU university

Deployment — Production

DigitalOcean Droplet · 159.65.152.72 · Ubuntu 24.04

Git repo	`/root/University-and-Course-data`
GitHub	github.com/Studyinfocentre/University-and-course-managment
Deploy	`git pull origin main` + `systemctl restart uni-api-py.service uni-celery.service`
FastAPI	`uni-api-py.service` (Uvicorn :8000), Nginx proxies `/api → 127.0.0.1:8000`
Celery	`uni-celery.service` (4 workers + beat)
Database	Local PostgreSQL, DB `university_portal`, owner `uniportal`

Critical: The DB URL must use `127.0.0.1`, NOT `localhost` — asyncpg's SSL hostname verification fails on this server's DNS. Apply all schema changes via direct `psql`; do NOT run alembic on prod (same DNS issue over TCP).

Monorepo Structure (pnpm workspaces)

Directory layout

- `artifacts/university-portal/ — React+Vite UI`
- `  src/pages, src/components, src/context`
- `backend-py/ — Python backend`
- `  app/main.py — FastAPI entry`
- `  app/routers/ — API handlers`
- `  app/services/scraper/ — engine`
- `    extractors/ — 15+ extractors`
- `    config/ — YAML config pkg`
- `    pipelines/ — PDF pipeline`
- `  scraper_config/unis/ — per-uni YAMLs`
- `  scripts/ — ops scripts`
- `  tests/ — pytest suite`

Key ops scripts `backend-py/scripts/`

- `bulk_approve.py` — batch promote staged courses
- `capture_baseline.py` — snapshot all unis
- `regression_sweep.py` — diff snapshots
- `bond_enrich.py` — Bond JSON API enrichment
- `csu_promotion_diagnostic.py`
- `week4_preflight.sh` / `week5_preflight.sh`
- `cricos_coverage_diagnostic.py`
- `check_gemini_model.py` — verify cost-optimal model
- `apply_migration_*.py` — schema migrations

Scraping Architecture — How It Works

From "trigger scrape" to a live course row — the full pipeline in `app/services/scraper/`

A scrape runs entirely inside a Celery worker. `orchestrator.run_scrape(university_id)` drives every phase below. Per-university behaviour is decided up front (Phase 0) and read by every extractor through a contextvar, so no site-specific logic is hard-coded in the engine.

0

Load configuration

`config/loader.py:load_uni_config() → config/context.py:set_uni_config()`

Slug is derived from the hostname (`www.acu.edu.au → acu`). The merged `UniConfig` is stored in a `ContextVar` ; extractors read it later via `require_uni_config()` .

▼

1

Discover course URLs

`discovery.py:discover_course_links()`

Tier 1 BFS crawl from `scrape_url` (`_LinkExtractor` + `_looks_like_course()` URL/anchor hints) → **Tier 2 sitemap** if <5 candidates or `always_sitemap_supplement` → **Tier 3 browser/Wayback** for JS-heavy or WAF-blocked sites. `_dedup_year_variants()` collapses `/2026/` vs `/2027/` URLs.

▼

2

Fetch each page

`http_fetcher.py | stealth_browser.py (if use_stealth_browser)`

Plain HTTP by default; falls back to a headless stealth browser for sites that block bots. Per-host URL rewriting (e.g. UNE `?international=true` , UOW `?students=international`) is applied here.

▼

3

Extractor cascade

`pipelines/single_course.py:extract_course() → extractors/*.py`

Rule-based extractors run first (`course_name` , `fee` , `intake` , `duration` , `english_test` , `location` , `cricos_code` ...). Each returns an `ExtractionResult` (normalized value + evidence). If fields are still missing, `ai_fallback.py` calls Gemini; `per_course_vision.py` OCRs fee-table images as a last resort.

▼

4

Staging gates & completeness

`stage_course.py:should_stage_course() → compute_completeness()`

`OnlineOnlyFilter` drops all-online courses (unless YAML opts out); `DomesticOnlyFilter` rejects "not available to international students" pages. `reject_if_missing` fields (e.g. `course_name`) are enforced. A 0–100 completeness score is computed for the auto-publish gate.

▼

5

Persist to staging

`scraped_courses + scraped_field_evidence + gemini_call_log`

Each course is written serially (to avoid DB deadlocks). `_persist_evidence()` stores source snippets per field for the UI "Evidence Review" modal. Old `pending` rows for the same URL from earlier jobs are deleted to prevent clutter.

▼

6

Approve & promote to live

`approve_course.py:approve_scraped_course() | scripts/bulk_approve.py`

Triggered by an operator or the auto-publish service (**85% completeness floor**). `resolve_sub_category()` snaps free-text categories to the canonical taxonomy. An upsert into the live `courses` table (+ satellite tables `fees` , `intakes` , `english_requirements`) matches on `(university_id, lower(name))` so re-runs update rather than duplicate.

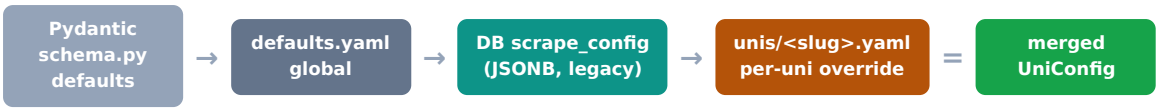
Why staging exists: scraping never writes directly to the live `courses` table. Everything lands in `scraped_courses` first, where it is scored, evidence-backed, and reviewed. Only an explicit approval (manual or auto-publish above the 85% floor) promotes data to live — this is what makes the AI-assisted scrape safe.

University YAML Config — Deep Dive

The mechanism that keeps per-university quirks out of the engine code

Every university scrapes differently — different page structures, fee formats, subdomains, and filtering needs. Instead of branching on hostname inside extractors, all of that lives in declarative YAML. Onboarding a new university means writing a small YAML file, **not** changing engine code.

Merge chain — lowest to highest priority



`config/loader.py:load_uni_config()` deep-merges these four sources. Later sources win, so a single field in `unis/uwa.yaml` overrides the global default without touching anything else. The result is set into a `ContextVar` (`config/context.py`) for the duration of the scrape job.

discovery: safe to replay on unknown unis

fallback_subdomains	Extra subdomains to probe (`handbook.{domain}`)
always_sitemap_supplement	Merge sitemap with BFS (JS SPAs)
block_url_patterns	Regex drop-list of URLs
allow_url_patterns	Regex whitelist of URLs
sitemap_url	Explicit sitemap override
use_wayback	archive.org fallback for WAF-blocked sites
bfs_page_budget	Override crawl page budget

extraction: site-specific, NOT replayed

fees.central_page	University-wide fee page / PDF URL
fees.default_currency	AUD / NZD per country
english.trust_vision_ocr	Disable OCR where it hallucinates
english.default_ielts/pte	Last-resort institutional standard
filters.domestic_only	Drop domestic-only courses
filters.online_only	Drop all-online (CSU opts out)
text_cleaning.location.strip_patterns	Regex noise removal
staging.reject_if_missing	Required fields to stage

Why the split matters: `discovery` settings make no assumptions about a specific site's content, so they are safe to replay in Tier-3 playbook matching against an unknown university. `extraction` settings encode site-specific knowledge and must never leak onto a different university — `UniConfig.for_tier3_replay()` strips the entire `extraction` section to enforce this.

Example — a minimal per-university override

```yaml
# scraper_config/unis/csu.yaml — Charles Sturt (distance-education heavy)
discovery:
  always_sitemap_supplement: true  # BFS misses paginated catalogue
extraction:
  filters:
    online_only:
      enabled: false  # keep CSU's online courses
```

Modification policy: `defaults.yaml` is the single highest-impact file — a change there affects EVERY university without an override. It requires a full regression sweep + human approval before merge. Per-uni YAML changes only re-run that one university, so they are safe after single-uni validation. **Always prefer a per-uni override over editing defaults.**